TP No3: Garbage collector

October 21, 2026



Un **ramasse miettes** (ou moins poétiquement garbage collector en anglais) est en mécanisme qui permet **de libérer les zones mémoires non utilisées par un processus**.

Les langages de haut niveau comme Java on des mécanismes de ramasse miettes.

Le langage C n'a pas de ramasse miettes. Le programmeur doit explicitement utiliser la fonction **free** pour libérer les zones mémoires qu'il n'utilise plus.

Il est possible d'ajouter un ramasse miettes au langage C: la bibliothèque libgc (GC de Boehm) fournit une nouvelle fonction **GC_MALLOC** qui lui permet de gérer les zones mémoire qu'il alloue, et de les libérer lorsqu'elles ne sont plus utilisées.

L'objectif de ce TP est de programmer un ramasse miette simplifié pour le langage C sur le modèle de celui développé par Hans-Juergen Bohem.

1 Garbage collector

Le but est de **remplacer l'appel à malloc par une fonction GC_malloc** qui gardera une trace des blocs alloués dans une liste chaînée.

Récupérez le fichier tinygc.c.Pour le compiler ajouter l'option **-Wno-unused-parameter** au compilateur pour supprimer les warnings relatifs aux paramètres non utilisés.

1. Chaque appel à la fonction GC_malloc appellera la fonction malloc du système et sauvegardera les adresses de début et de fin de la zone allouée dans une liste.
2. la fonction GC_collect parcourra tous les mots de la pile à la recherche d'adresses :
3. si le mot correspond à une adresse contenue dans une zone allouée, la zone sera marquée comme utilisée,

Après avoir parcouru la pile, les zones non marquées seront libérées par un appel à la fonction free du système.

1.1 Récupérer les informations sur les blocs

La liste des blocs alloués sera conservée **dans une liste chaînée dont l'élément est la structure cell**. Cette structure cell contient l'adresse de début d'un bloc alloué, la taille, un flag d'utilisation et un pointeur sur l'élément suivant.

```
typedef struct cell {
    address_t start;
    size_t size;
    int flags;
    struct cell *next;
} cell_t;
```

La liste des blocs alloués sera stockée dans une variable globale BLOCKS:

```
cell_t *BLOCKS = NULL;
```

Pendant le développement, il sera utile de pouvoir afficher la liste des zones allouées.

Complétez la fonction **print_list** dans le fichiers tinygc.c.

```
// display a visual representation of a list
void print_list(cell_t *list) {
    int index = 0;
    fprintf(stderr, "  +-------+------------------+------------+------+\n");
    fprintf(stderr, "  | index |    address       |    size    | used |\n");
    fprintf(stderr, "  +-------+------------------+------------+------+\n");
    cell_t *p;
    for (???; ???; ???) {
        fprintf(stderr, "  |  %03i  |  %10p  |  %6lu  |  %c   |\n",
                        index,
                        ???,
                        ???,
                        ???);
        n++;
    }
    fprintf(stderr, "  +-------+------------------+------------+------+\n");
}
```

N'oubliez pas de tester. Vous pouvez pour ceci utiliser la fonction insert_BLOCKS qui insère des valeurs dans la liste (globale) BLOCKS.

```
void insert_BLOCKS(address_t start, size_t size);
```

Complétez la fonction GC_malloc qui appelle la fonction malloc du système et insère les informations pertinentes dans la liste BLOCKS grâce à la fonction insert_BLOCKS.

1.2 Marquer les blocs utilisés

Nous allons maintenant écrire la fonction mark_region qui parcourt la mémoire entre les adresses start et end, et pour chaque mot trouvé si il contient une adresse qui correspond un bloc de données marque ce bloque pour dire qu'il est utilisé.

```
// mark blocks from a memory region
void mark_region(address_t start, address_t end) {
    address_t p;  // pointer to memory
    address_t v;  // value at pointer p
    for (p=start; p<end; p+=WORD_SIZE) {
        v = *p;
        mark_BLOCK(v);
    }
}
```

Si vous compilez ce code, le compilateur affichera un warning pour la ligne

```
v = *p;
```

Ajoutez le cast pertinent pour corriger ce problème. (Attention, seul le typage de cette ligne est incorrect : il ne faut pas modifier son contenu "calculatoire" en changeant les variables ou en ajoutant / supprimant des ou &.)

L'appel mark_BLOCK(v) parcourt la liste de zones allouées BLOCKS et marque la zone qui contient v comme utilisée (bit de poids faible du champs flags à 1).

Complétez cette fonction.

1.3 Marquer les blocs utilisés dans la pile

La fonction mark_from_stack parcourt la pile pour marquer les blocs correspondants. Pour ceci, il faut appeler la fonction mark_region avec les arguments start et end appropriés.

La variable globale STACK_TOP contient l'adresse du haut de la pile. Elle doit être initialisé dans la fonction main.

En vous inspirant des tests faits dans la première partie du TP (affichages d'adresses de la pile), initialisez la variable STACK_TOP de manière appropriée. (Attention, cette variable doit être alignée sur un mot. Le plus simple est d'utiliser une variable de type int pour garantir ceci...)

L'adresse du bas de la pile change au cours de l'exécution, au fil des déclarations de variables et d'appels de fonctions.

Comment peut-on trouver l'adresse du bas de la pile à l'intérieur de la fonction mark_from_stack ?

Appelez la fonction mark_region avec les bonnes valeurs de start et end.

N'oubliez pas de commenter ce que vous faites !

2 Exécuter le garbage collector

La fonction GC_collect (fournie) appelle la fonction mark_from_stack et libère toutes les zones non utilisées.

Vérifiez que votre ramasse miettes fonctionne correctement en ajoutant un test dans la fonction principale qui :

1. fait quelques allocations avec GC_malloc,
2. écrase certains pointeurs avec, par exemple, des NULL,
3. appelle la fonction print_list,
4. appelle la fonction GC_collect,
5. appelle la fonction print_list.

Si tout fonctionne, la seconde liste devrait contenir moins de zones allouées...

3 Parcours du tas

On peut initialiser un tableau dynamique à deux dimensions avec

```
int **T = malloc(size * sizeof(int*));
for (int i=0; i<size; i++) {
  T[i] = malloc(size * sizeof(int));
}
```

Remplacez les appels à malloc par des appels à GC_malloc et regarder ce que fait GC_collect. Qu'en pensez vous ?

Proposez une solution et programmez une fonction mark_from_heap qui sera appelé depuis GC_collect pour corriger le problème.